

TRAINING CHESS BOTS WITH VARIOUS MACHINE LEARNING TECHNIQUES

by

BHAVYA SINGH
URN: 6839571

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Dr Joey Sik Chun Lam

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Bhavya Singh
May 2026

© Copyright Bhavya Singh, May 2026

Abstract

Write a summary of the work presented in your dissertation. Introduce the topic and highlight your main contributions and results. The abstract should be comprehensible on its own, and should not contain any references. As far as possible, limit the use of jargon and abbreviations, to make the abstract readable by non-specialists in your area. Do not exceed 300 words.

Acknowledgements

Write any personal words of thanks here. Typically, this space is used to thank your supervisor for their guidance, as well as anyone else who has supported the completion of this dissertation, for example by discussing results and their interpretation or reviewing write ups. It is also usual to acknowledge any financial support received in relation to this work.

Contents

1	Introduction	15
1.1	Problem Statement	15
1.2	Motivation	15
1.3	The Challenge of Chess	16
1.3.1	Computational Complexity	16
1.3.2	Failure of Naive Approaches	16
1.4	Critique of Existing Solutions	16
1.5	Overview of My Approach	16
1.5.1	Machine Learning Architectures	16
1.5.2	Comparison Methodology	17
1.6	Summary of Key Results	17
1.7	Scope and Limitations	17
1.8	Aims and Objectives	17
1.9	Report Structure	18
2	Literature Review	19
2.1	Classical Chess Engines	19
2.2	Probabilistic Search Methods	19
2.3	The Deep Learning Revolution	20
2.4	Supervised Learning Approaches	20
2.5	Recent Innovations	20

2.6	Game AI Frameworks	21
2.7	Web-Based Chess Platforms	21
2.7.1	Existing Interactive Chess Platforms	21
2.7.2	Bot-vs-Bot Comparison Platforms	21
2.7.3	Human-vs-Bot Interfaces	21
2.7.4	Web Technologies for Real-Time Games	21
2.8	Critical Analysis	22
2.9	Summary	23
3	Methodology and Design	24
3.1	Requirements Analysis	24
3.1.1	Functional Requirements	24
3.1.2	Non-Functional Requirements	25
3.2	Framework Architecture	25
3.2.1	Chess Environment Module	25
3.2.2	AI Agent Module	26
3.2.3	Training and Evaluation Engine	27
3.3	Design Decisions and Justification	27
3.3.1	Choice of Python and PyTorch	27
3.3.2	Backbone-Head Decoupling	28
3.3.3	Streaming Data Pipeline	28
3.3.4	Move Encoding	28
3.3.5	Dual-Headed AlphaZero-Style Architecture	28
3.3.6	Enhanced Encoder Design	29
3.3.7	Spatial Policy Head	29
3.3.8	Squeeze-and-Excitation Blocks	30
3.3.9	Data Pipeline Evolution	30

3.4	Chess Implementation	31
3.5	Web Platform Design	31
3.5.1	Platform Requirements	31
3.5.2	System Architecture	32
3.5.3	Game Modes	32
3.5.4	User Interface Design	32
3.5.5	Backend API Design	32
3.6	Technical Challenges	33
4	Implementation	34
4.1	Development Environment and Tools	34
4.2	Core Framework Implementation	35
4.2.1	Abstract Base Classes	35
4.2.2	Factory Pattern	35
4.2.3	Configuration System	35
4.3	Supervised Learning Model	36
4.3.1	Data Preparation	36
4.3.2	Neural Network Architecture	37
4.3.2.1	Convolutional Neural Networks	37
4.3.2.2	Transformers	37
4.3.2.3	Graph Neural Networks	37
4.3.3	Training Process	38
4.3.3.1	Loss Functions	38
4.3.3.2	Optimisation	38
4.3.3.3	Mixed Precision and Compilation	38
4.3.3.4	Checkpointing	38
4.4	Architectural Enhancements	39

4.4.1	Enhanced CNN Encoder	39
4.4.2	Spatial Policy Head	39
4.4.3	Squeeze-and-Excitation Blocks	39
4.4.4	Flash Attention	40
4.4.5	Activation Functions	40
4.5	Data Pipeline Evolution	40
4.5.1	Stage 1: Deduplication	40
4.5.2	Stage 2: PV Line Unrolling	40
4.5.3	Stage 3: Tablebase Enrichment	41
4.5.4	Stage 4: Target Computation and Cleanup	41
4.5.5	Stage 5: Binary Sharding	41
4.6	Baseline Models	41
4.7	Benchmarking and Evaluation Pipeline	42
4.8	Testing and Debugging	42
4.9	Documentation	43
5	Evaluation	44
5.1	Evaluation Methodology	45
5.2	Supervised Learning Model Results	45
5.2.1	Training Performance	45
5.2.2	Playing Strength	45
5.2.3	Error Analysis	45
5.3	Comparative Analysis	45
5.4	Ablation Studies	45
5.5	Web Platform Evaluation	45
5.5.1	Usability Testing	45
5.5.2	Performance and Responsiveness	45

5.5.3	Bot-vs-Bot Match Results via Platform	45
5.6	Comparison with Related Work	45
5.7	Interpretation and Significance	45
5.8	Limitations and Challenges	45
6	Critical Review and Reflection	46
6.1	Achievement of Objectives	46
6.2	What Worked Well	46
6.3	What Didn't Work Well	47
6.4	Lessons Learned	48
6.5	Personal Reflection	48
6.6	Future Work	48
6.6.1	Reinforcement Learning	48
6.6.2	Web Platform	49
6.6.3	Long-term Research Directions	49
6.7	Broader Impact	50
7	Legal, Social, Ethical, and Professional Issues	51
7.1	Ethical Considerations	51
7.1.1	Research Ethics	51
7.1.2	Responsible AI	51
7.2	Legal Issues	52
7.2.1	Intellectual Property	52
7.2.2	Copyright and Licensing	52
7.3	Social Issues	52
7.3.1	Accessibility	52
7.3.2	Web Platform Accessibility	52
7.3.3	Impact on Chess Community	53

7.4	Professional Issues	53
7.4.1	Code of Conduct	53
7.4.2	Information Security	53
7.4.3	Web Platform Security	53
7.5	Sustainability	54
7.5.1	Environmental Impact	54
7.5.2	UN Sustainable Development Goals	54
7.6	Summary	54
8	Conclusion	56
A	Code Listings	58
B	Additional Results	59
C	User Guide	60
D	Game Examples	61
E	Ethical Approval Documentation	62
F	Project Planning Documents	63

List of Figures

List of Tables

4.1	Key libraries and their roles in the framework.	34
-----	---	----

Glossary

A_f	The source message, being a sequence of f source symbols $a_1a_2 \dots a_f$
a_i	The i^{th} symbol in the source message, where $a_i \in S_m$
B_g	The decoded message, being a sequence of g source symbols $b_1b_2 \dots b_g$
b_i	The i^{th} symbol in the decoded message, where $b_i \in S_m$
C_h	The transmitted (compressed) message, being a sequence of h Tunstall codewords $c_1c_2 \dots c_h$
c_i	The i^{th} codeword in the transmitted message, where $c_i \in T_n$
D_h	The received (compressed) message, which for a complete Tunstall code is a sequence of h Tunstall codewords $d_1d_2 \dots d_h$
d_i	The i^{th} codeword in the received message, where $d_i \in T_n$ for a complete Tunstall code

Abbreviations

BER	Bit Error Rate
BPSK	Binary Phase Shift Keying
BSC	Binary Symmetric Channel
DCT	Discrete Cosine Transform
ECC	Error Correcting Codes
FEC	Forward Error Correction
JPEG	Joint Photographic Experts Group
MPEG	Moving Pictures Experts Group
SER	Symbol Error Rate
SNR	Signal to Noise Ratio

Chapter 1

Introduction

1.1 Problem Statement

For decades, the game of chess has been a benchmark of Artificial Intelligence progress — a controlled environment used to measure the evolution of machine logic. Early milestones, such as IBM's Deep Blue, relied on carefully handcrafted heuristics and "brute force" calculations. While effective, these systems were inherently restrictive, struggling to generalize beyond the specific rules encoded by their creators.

Today, the landscape is dominated by neural-heavy architectures like AlphaZero, MuZero, and Leela Chess Zero. These engines have pushed ELO ratings beyond the 3500 mark, effectively concluding the era of human-computer competition. However, this superiority comes at a cost: modern engines are increasingly reliant on massive computational overhead and deep search trees to maintain their edge.

1.2 Motivation

The current state-of-the-art engines are outstanding, but they are also "computationally greedy." Their strength is acquired from exploring millions of potential future states (search) before making a move. This reliance on search constitutes a difficult obstacle to entry for mobile or edge-computing applications.

The motivation lies in the pursuit of "Search-less Chess." By leveraging Convolutional Neural Networks (CNNs), Transformers, and Graph Neural Networks (GNNs). We aim to investigate whether a model can develop a "grandmaster's intuition"—the ability to look at a board and immediately identify the optimal move without the safety net of a search algorithm. This would not only drastically reduce move latency but also offer insights into how deep learning architectures internalize complex spatial and strategic relationships.

1.3 The Challenge of Chess

Chess is a game of perfect information, yet it remains computationally "unsolved" because of its high strategic depth. A player must simultaneously balance tactical immediate threats (checks, captures) with abstract long-term goals (piece mobility, pawn structure, and king safety).

1.3.1 Computational Complexity

The complexity of chess is often illustrated by Shannon's Number, which estimates the game tree complexity at approximately 10^{120} . Even the state space—the number of possible legal positions—is estimated at 10^{40} . Because the number of possible variations grows exponentially with every half-move (ply), brute-force calculation is a physical impossibility. This necessitates an approach that can prune the search space or, as we propose, bypass it entirely through high-fidelity policy prediction.

1.3.2 Failure of Naive Approaches

Simple heuristics (e.g., assigning a point value to pieces) fail because chess is highly contextual; a Queen is worthless if the King is in an inescapable checkmate. Naive machine learning models often fall into the trap of "greedy" play—capturing material while ignoring a looming positional collapse. Without the "look-ahead" provided by search, a model must be significantly more sophisticated to recognize the subtle cues that distinguish a winning position from a losing one.

1.4 Critique of Existing Solutions

Current engines like Stockfish 18 are incredibly powerful but function as "black boxes" of search optimization. They require significant CPU/GPU resources to reach their peak ELO. Furthermore, even hybrid models like Leela Chess Zero—which use neural networks for evaluation—still rely on Monte Carlo Tree Search (MCTS) to validate their "hunches."

1.5 Overview of My Approach

1.5.1 Machine Learning Architectures

This project explores an architectural comparison to determine which structure best captures the tactical and strategic nuances of chess without search.

- **CNNs:** To treat the board as a 2D image, capturing local spatial patterns and piece interactions.
- **Transformers:** To leverage self-attention mechanisms, allowing the model to understand long-range dependencies across the board (e.g., a Bishop's influence from a8 to h1).
- **GNNs:** To model the board as a dynamic graph where pieces are nodes and their legal moves are edges, emphasizing the relational geometry of the game.

We will train these models using the Lichess Stockfish dataset, effectively "distilling" the knowledge of a 3500-ELO engine into our search-less architectures.

1.5.2 Comparison Methodology

To validate our approach, we will benchmark our models against Stockfish at varying "depth" constraints. This allows us to simulate different ELO ranges and see exactly where a search-less model begins to falter compared to a searching engine. Additionally, we will use the Lichess Puzzle dataset to test the models on specific tactical motifs, providing a granular look at their "chess IQ" in areas like forks, pins, and endgame transitions.

1.6 Summary of Key Results

1.7 Scope and Limitations

The goal of this research is not to compete with the world's strongest engines, but to push the ceiling of what is possible within the "search-less" paradigm.

Limitations: We acknowledge that without search, our models may struggle with "horizon effects," where a tactic requires a deep calculation beyond the model's immediate pattern recognition. Similarly, highly technical endgames (e.g., Rook, and Bishop vs. Rook) may remain a challenge for a purely intuitive model.

1.8 Aims and Objectives

The primary objective is to engineer and evaluate a suite of deep learning models (CNN, Transformer, GNN) capable of achieving a 2000 ELO rating without the use of search algorithms. Success will be measured by:

1. **Inference Speed:** Demonstrating a significant reduction in move generation time compared to traditional engines.

2. **Generalization:** Proving the model has "learned" chess principles rather than memorized positions, verified via performance on unseen puzzle sets.
3. **Comparative Analysis:** Identifying which architecture (spatial, relational, or sequential) is best suited for the unique geometry of chess.

1.9 Report Structure

The remainder of this report is organized as follows: **Chapter 2** surveys the history of chess AI and the technical evolution of deep learning in games. **Chapter 3** details our methodology, from data curation and normalization to the specific hyperparameters of our three architectures. **Chapter 4** presents a rigorous analysis of the results, including ELO benchmarking and tactical performance. Finally, **Chapter 5** discusses the implications of these findings for the future of lightweight, intuitive AI.

Chapter 2

Literature Review

2.1 Classical Chess Engines

Earlier Chess engines relied heavily on the minimax algorithm combined with alpha-beta pruning to efficiently search the game tree.

These engines utilized handcrafted evaluation functions that assessed board positions based on material count, piece activity, king safety, and other heuristics Example Deep Blue.

While effective, these approaches had limitations in terms of adaptability and often required extensive domain knowledge to fine-tune the evaluation parameters. Stockfish, one of the most powerful open-source chess engines, exemplifies this classical approach with its highly optimized search algorithms and evaluation function.

2.2 Probabilistic Search Methods

Monte Carlo Tree Search (MCTS) is a probabilistic search method that has been successfully applied to various games, including chess. MCTS builds a search tree based on random sampling of the game space, allowing it to explore promising moves without exhaustive search. This approach has advantages over traditional minimax algorithms, particularly in complex game states where the search space is vast. MCTS has been used in engines like Leela Chess Zero, which combines MCTS with neural network evaluations to achieve strong performance.

Additionally, MCTS can adapt its search strategy based on the outcomes of previous simulations, making it more flexible in dynamic game environments. This adaptability has contributed to its success in various game AI applications beyond chess, such as Go and general game playing.

2.3 The Deep Learning Revolution

AlphaZero, developed by DeepMind, marked a significant breakthrough in chess AI by combining deep learning with self-play reinforcement learning. AlphaZero’s architecture consists of a policy network that predicts the probability of winning moves and a value network that estimates the expected outcome of a position. Through self-play, AlphaZero was able to learn and improve its performance without relying on human game data, achieving superhuman performance in chess, shogi, and Go.

This approach has inspired the development of open-source implementations like Leela Chess Zero, which utilizes a similar architecture and training methodology to achieve competitive performance against traditional engines. The success of deep learning in chess has also spurred research into more advanced neural network architectures and training techniques, further pushing the boundaries of what AI can achieve in strategic games.

2.4 Supervised Learning Approaches

DeepChess uses a two-stage deep neural network: a Pos2Vec autoencoder pre-trained unsupervised on 640K human games from the CCRL database, followed by a supervised comparison network that takes two positions as input and outputs which is more favourable. The system never assigns numerical evaluations — it directly learns to rank positions, reaching grandmaster-level play with no hand-crafted heuristics.

Giraffe trains a neural network evaluation function using temporal-difference learning via self-play, with minimal hand-crafted input features. Despite receiving no explicit human chess knowledge, the resulting engine plays at approximately FIDE International Master level *top2.2% of rated players*, comparable to the strongest handcrafted engines of the time. Key distinctions from DeepChess: Giraffe uses self-play (not human game databases) and TD-learning rather than supervised pairwise classification.

2.5 Recent Innovations

Graph representations of chess positions have been explored to capture the relational structure of the game more effectively. Tomas Rigaux and Hisashi Kashima (2024) proposed a graph-based neural network architecture that models the interactions between pieces and their positions on the board, leading to improved evaluation accuracy and strategic understanding.

Explainable AI (XAI) has also gained attention in chess, with researchers like Svendsen and Tørresen (2022) developing methods to provide insights into the decision-making processes of chess engines. This includes techniques for visualizing the influence of different pieces and moves on the engine’s evaluations, helping players

and researchers understand the underlying strategies employed by AI agents.

2.6 Game AI Frameworks

The library provides move generation and validation, PGN parsing and writing, Polyglot opening book reading, Syzygy/Gaviota tablebase probing, and full UCI/XBoard engine communication - essentially everything needed to interface with any standard chess engine in Python.

2.7 Web-Based Chess Platforms

2.7.1 Existing Interactive Chess Platforms

Thibault Duplessis (lead dev), Lichess / Lila — open-source chess server. Lichess is written in Scala/Play with a fully asynchronous architecture using Akka actors, MongoDB for game storage (800M+ games), Elasticsearch indexing, and nginx proxying both HTTP and WebSocket connections. The real-time move streaming is handled natively via WebSockets.

2.7.2 Bot-vs-Bot Comparison Platforms

Chessprogramming Wiki — "Tournament Manager" entries and TCEC (Top Chess Engine Championship) as a descriptive reference for bot-vs-bot spectator design. Search "TCEC chess engine competition" for the definitive real-world example of a spectator-oriented bot-arena platform.

2.7.3 Human-vs-Bot Interfaces

Lichess API Board Endpoints — `boardGameStream` / `boardGameMove` provide the reference implementation for streaming live moves from a bot to a human player in real time. The API is REST+NDJSON.

2.7.4 Web Technologies for Real-Time Games

"The WebSocket Protocol." IETF RFC 6455, 2011. The normative standards reference — cite when defining the protocol your game communication layer uses. Available at tools.ietf.org/html/rfc6455.

Using Go and Svelte for the frontend and backend respectively is a novel choice that offers performance benefits and a modern development experience. Go's concurrency model is well-suited for handling multiple simultaneous games and real-time interactions, while Svelte's reactive framework allows for efficient UI updates with

minimal overhead. This combination is less common in game development compared to traditional JavaScript frameworks.

2.8 Critical Analysis

Exploring the Latest Neural Network Architectural Developments in Chess AI," 2024 — confirms CNNs remain competitive in hybrid $NNUE + \alpha - \beta$ deployments and are the most computationally efficient at inference time.

CNNs have a limited receptive field: a 3x3 kernel only sees adjacent squares, requiring many stacked layers to propagate information across the board. This is structurally mismatched to chess, where a single piece *e.g.*, *arookorqueen* influences the entire board instantly.

Primary citation for the Square Transformer paradigm. Chessformer (CF-240M) achieves 2347 Elo and 93.5% puzzle accuracy at 30x fewer FLOPs than comparable AlphaZero-scale models. The critical finding is that positional encoding is the decisive design choice — absolute, relative, and dynamic *Shawetal.* encodings produce markedly different results.

Trains a large transformer on 10 million Lichess games annotated with Stockfish 16 at 50ms per position — architecturally the closest existing work to your own pipeline. Discusses action-value and state-value heads and the tradeoffs between square-based and piece-based tokenisation strategies. Critical gap your work addresses: Ruoss et al. use a single architecture; you systematically compare Square and Piece Transformer under identical conditions.

Quadratic attention complexity over 64 tokens is manageable but costly at scale compared to CNN inference.

Transformers require careful positional encoding — absolute embeddings fail to capture board geometry and naive relative encodings lose piece-type information.

AlphaGateau (GAT + GATEAU edge-feature layer) outperforms ResNet-based AlphaZero by an order of magnitude in sample efficiency on a 5x5 chess variant, demonstrating that graph representations encode relational structure (legal moves as edges) that grids cannot express compactly.

Rigaux & Kashima train only on a 5x5 variant and fine-tune to 8x8 — no published paper presents a full GCN/GAT chess engine trained end-to-end on standard chess from scratch against a CNN and Transformer baseline on the same dataset. This is one of the most direct gaps your project fills.

2.9 Summary

Representation matters more than depth — Chessformer shows position encoding is the decisive variable for transformers; Rigaux & Kashima show graph edges (legal moves) encode relational structure that grids miss. Your multi-architecture comparison is the natural empirical test of this claim.

Supervised Stockfish annotation is a viable and scalable training signal — Ruoss et al., Molinelli et al., and Oshri & Khandwala all validate this pipeline, but none investigate PV unrolling for position diversity.

No fair cross-architecture benchmark exists — every existing paper proposes one architecture in isolation. Your project’s controlled comparison on identical Lichess/Stockfish data with PV-unrolled positions is the gap that makes it publishable.

Chapter 3

Methodology and Design

3.1 Requirements Analysis

The framework is designed to satisfy a set of functional and non-functional requirements that collectively enable a fair, controlled comparison of deep learning architectures for search-less chess.

3.1.1 Functional Requirements

1. **Multi-Architecture Support:** The framework must support training and inference across at least three distinct neural network families—CNNs, Transformers, and GNNs—under identical data and evaluation conditions.
2. **Pluggable Backbone–Head Composition:** Any backbone (e.g., ResNet, GAT) must be composable with any output head (Policy, Value, or Dual) without code duplication.
3. **Supervised Training:** The system must ingest engine-annotated positions from Parquet files and train models using both hard and soft policy labels alongside centipawn-derived value targets.
4. **Scalable Data Pipeline:** The system must support a multi-stage data preparation pipeline capable of processing over one billion positions, including deduplication, PV line unrolling, tablebase enrichment, and offline pre-encoding into binary shards.
5. **Benchmarking:** An automated pipeline must pit trained models against Stockfish at configurable depths, recording per-move evaluations and exporting PGN files for post-hoc analysis.
6. **Agent Abstraction:** A common agent interface must allow any model, engine, or random baseline to participate in games interchangeably.

3.1.2 Non-Functional Requirements

1. **Performance:** Training must leverage mixed-precision arithmetic (float16) and `torch.compile` where applicable to maximise GPU throughput.
2. **Memory Efficiency:** The data pipeline must stream from disk one row-group at a time, bounding peak RAM usage regardless of dataset size.
3. **Modularity:** Each subsystem (encoders, models, training loops, agents) must reside in its own package with clearly defined interfaces, facilitating independent development and testing.
4. **Reproducibility:** All hyperparameters must be specified in a single YAML configuration file, and training checkpoints must be restorable to enable exact experiment replay.
5. **Extensibility:** Adding a new backbone or encoder should require implementing a single abstract class without modifying existing training or evaluation code.

3.2 Framework Architecture

The framework is organised into five principal modules, each encapsulating a distinct concern.

3.2.1 Chess Environment Module

The chess environment module wraps the `python-chess` library to provide board management and move encoding. A `BoardWrapper` class exposes properties for legal moves, game-over detection, and piece queries, while a statically generated lookup table maps every possible UCI move string (including promotions) to a unique integer index. This table contains 4544 entries and serves as the universal action space for all policy heads.

Three specialised state encoders translate a `chess.Board` object into the tensor format expected by each architecture family:

- **CNNEncoder:** In its initial form, the encoder produces an $18 \times 8 \times 8$ tensor with 12 bitboard planes (six piece types for each colour), four constant planes for castling rights, one plane for the en passant square, and one plane encoding the side to move as +1 (White) or -1 (Black). The board is always encoded from the current player’s perspective by flipping ranks and files when Black is to move. The encoder was subsequently extended with tactical feature planes and material imbalance channels (described in Section 3.3.6).

- **GNNEncoder:** Constructs a graph with 64 nodes (one per square). Each node carries a 14-dimensional feature vector: a 13-element one-hot piece identifier and a colour scalar. Edges are derived from two sources—physical adjacency (King move patterns) and tactical relations (`board.attackers`)—and carry three-dimensional attributes: normalised Manhattan distance, a binary ray-tracing flag indicating an unblocked line of influence, and a binary legality flag. Reverse edges are added to enforce structural symmetry while preserving directional legality features.
- **TransformerEncoder:** Supports two tokenisation schemes. The **SquareTokenizer** emits a fixed-length sequence of 64 tokens (one per square) with a 13-class vocabulary (empty plus twelve piece types), normalised rank/file coordinates, and full attention. The **PieceTokenizer** emits a variable-length sequence of up to 32 tokens (one per piece), ordered by colour (current player’s pieces first), with each token carrying its piece type and square position. Padding masks ensure that empty slots do not contribute to attention.

All three encoders inherit from an abstract **StateEncoder** base class that mandates `encode()`, `get_input_shape()`, and `name` methods, enabling the factory layer to select the correct encoder at runtime based on the chosen backbone.

3.2.2 AI Agent Module

The agent module defines a common interface through the abstract **ChessAgent** class, which requires every agent to implement `get_move(board, time_limit)` and a `name` property. Four concrete agents are provided:

- **RandomAgent:** Selects uniformly at random from the legal move set. Serves as the lower-bound baseline.
- **LearningAgent:** Wraps any trained **ChessModel** and its corresponding encoder. At inference time, it encodes the board, runs a forward pass, masks illegal moves by setting their logits to $-\infty$, and selects the move via greedy argmax or temperature-controlled sampling with optional top- k filtering.
- **MCTSAgent:** Implements AlphaZero-style Monte Carlo Tree Search. Each node stores visit counts, cumulative value, and a prior probability from the policy network. Selection follows the PUCT formula $UCB = Q + c_{\text{puct}} \cdot P \cdot \frac{\sqrt{N_{\text{parent}}}}{1+N}$. Leaf nodes are evaluated by the value head, and values are back-propagated with sign flips to account for alternating perspectives. Dirichlet noise ($\alpha = 0.3$, $\varepsilon = 0.25$) is optionally added at the root for exploration during self-play data generation.
- **UCIAgent:** Communicates with any UCI-compatible engine (e.g., Stockfish) via `python-chess`’s engine protocol. Supports configurable depth, time limits, skill levels, and MultiPV analysis for obtaining centipawn-scored move distributions.

3.2.3 Training and Evaluation Engine

The training subsystem centres on the `Trainer` class, which orchestrates the supervised training loop. Key features include:

- **Loss Functions:** A factory selects the appropriate loss based on the head type. `PolicyLoss` computes KL divergence against soft label distributions or cross-entropy against hard labels with optional label smoothing. `ValueLoss` uses mean squared error between the predicted tanh-bounded value and the target. `DualLoss` combines both with configurable weights.
- **Mixed Precision:** The trainer wraps the forward pass in `torch.autocast` (float16 on CUDA/MPS) and applies `GradScaler` on CUDA to avoid underflow in low-precision gradients.
- **Compilation:** Non-GNN backbones are passed through `torch.compile` for graph-level kernel fusion and optimised execution.
- **Scheduling:** Cosine annealing and step-decay learning rate schedules are supported, configured via YAML.
- **Checkpointing:** Model weights, optimiser state, scheduler state, epoch counter, and best validation loss are serialised to disk after every epoch. Intermediate checkpoints are pruned to retain only every fifth epoch, the best, and the final model.

As training progressed, the initial streaming Parquet pipeline was superseded by a multi-stage offline data preparation system designed to handle over one billion positions. This enhanced pipeline is described in Section 3.3.9.

3.3 Design Decisions and Justification

3.3.1 Choice of Python and PyTorch

Python was selected for its dominance in the machine learning ecosystem and the maturity of supporting libraries (`python-chess`, `pyarrow`, `torch_geometric`). PyTorch was preferred over TensorFlow for its eager-mode debugging, first-class support for dynamic computation graphs (essential for the variable-length PieceTransformer and graph-structured GNN inputs), and its `torch.compile` JIT pathway for production-grade throughput.

3.3.2 Backbone–Head Decoupling

A deliberate design choice separates backbone feature extraction from task-specific output heads. Every backbone subclasses `ChessModel` and implements `forward_backbone()` to produce a fixed-dimension feature vector. The `set_head()` method then attaches one of three interchangeable heads. This decoupling yields $6 \times 3 = 18$ valid backbone–head combinations from six backbone and three head implementations, without any combinatorial code explosion. More importantly, it ensures that when comparing architectures, the only variable is the backbone; the output layer, loss function, and training loop remain identical.

3.3.3 Streaming Data Pipeline

The dataset is stored in Apache Parquet format, which supports columnar compression and row-group-level random access. The `ChessDataset` class, implemented as a PyTorch `IterableDataset`, reads one row-group at a time and shuffles within each group. This design bounds peak memory to the size of a single row-group (typically 64–128 MB) regardless of total dataset size, which is critical for scaling to hundreds of millions of positions. Multi-worker data loading is supported by partitioning row-groups across workers using their `worker_id`.

3.3.4 Move Encoding

Rather than the $8 \times 8 \times 73$ spatial policy planes used by AlphaZero, a flat vector of 4544 logits is used. Each index corresponds to a unique UCI move string, generated exhaustively from all valid (`from_square`, `to_square`, `promotion`) tuples. This representation is architecture-agnostic: the same policy head attaches to CNNs, Transformers, and GNNs without modification. The trade-off is the loss of spatial locality in the policy output, which the flat head compensates for with a hidden layer.

3.3.5 Dual-Headed AlphaZero-Style Architecture

The default configuration uses a dual-headed model that jointly predicts the policy distribution and the board evaluation from shared backbone features. This follows the AlphaZero paradigm where the value head provides a training signal for the backbone to learn positional understanding, while the policy head learns tactical move selection. The combined loss $\mathcal{L} = \lambda_p \cdot \mathcal{L}_{\text{policy}} + \lambda_v \cdot \mathcal{L}_{\text{value}}$ is weighted equally by default ($\lambda_p = \lambda_v = 1.0$).

3.3.6 Enhanced Encoder Design

After initial experiments with the base 18-channel CNN encoder, the encoder was extended with explicit tactical feature planes and material imbalance channels to inject domain knowledge that the model would otherwise need to discover from raw piece positions alone.

Tactical Feature Planes: The following additional 8×8 planes were added:

- **Attack/Defence Maps (4 planes):** Per-square counts of attackers and defenders for each side, normalised by the maximum count. These surfaces make piece safety immediately visible to the first convolutional layer.
- **Mobility (2 planes):** The number of legal moves available to each piece, broadcast to its square. This encodes piece activity without requiring the model to trace move generation logic.
- **Pin Detection (1 plane):** Binary flags for pieces that are absolutely pinned to their King, identifying pieces whose movement is constrained.
- **King Safety (2 planes):** Heuristic scores for each side’s King based on pawn shield integrity and open files near the King.

Material Imbalance (5 planes): Rather than a single scalar material difference, five constant 8×8 planes encode the per-piece-type count difference (our count minus their count) for Pawns, Knights, Bishops, Rooks, and Queens independently. This representation allows the model to distinguish strategically different material configurations—for example, two Knights versus a Rook—that a scalar sum would conflate.

The extended encoder thus produces a $32 \times 8 \times 8$ tensor (18 base + 9 tactical + 5 material), with the additional channels computed directly from the `python-chess` board state.

3.3.7 Spatial Policy Head

The initial policy head applied global average pooling to the backbone’s spatial feature maps before projecting to the 4544-move logit vector. This discards square-level information critical for move selection (i.e., which piece on which square should move where). The spatial policy head replaces global average pooling with a 1×1 convolution that reduces the channel dimension, followed by a flatten operation that preserves the 8×8 spatial grid. The resulting $8 \times 8 \times C'$ representation is then projected to the move logits via a linear layer, allowing the network to associate specific board locations with specific moves.

3.3.8 Squeeze-and-Excitation Blocks

Squeeze-and-Excitation (SE) blocks were added to the ResNet backbone’s residual blocks. Each SE block applies global average pooling to “squeeze” the spatial dimensions into a channel descriptor, passes it through a two-layer fully connected bottleneck with ReLU activation, and uses the resulting channel weights to “excite” (scale) the original feature maps via element-wise multiplication. This channel attention mechanism allows the ResNet to dynamically prioritise different feature channels depending on the position—for example, emphasising pawn structure planes in endgames and King safety planes in middlegames. The SE blocks are placed inside the residual blocks so that the skip connections maintain gradient stability.

3.3.9 Data Pipeline Evolution

The initial streaming Parquet pipeline served well for prototyping but became a bottleneck at scale: the runtime board encoding via `python-chess` was CPU-bound and could not saturate GPU training. A five-stage offline data preparation pipeline was designed to handle over one billion positions:

1. **Deduplication (DuckDB):** FEN strings are normalised (stripping move counters) and grouped using DuckDB’s out-of-core SQL engine. For duplicate positions, the analysis with the highest depth or node count is retained.
2. **PV Unrolling:** For each base position, the first N moves of the principal variation are played out. Each intermediate position inherits the root evaluation (with sign flips for alternating sides), avoiding the cost of re-evaluating with Stockfish. Drift analysis confirmed that 96% of inherited evaluations remain within ± 100 centipawns of the true value.
3. **Tablebase Enrichment:** Positions with five or fewer pieces are probed against Syzygy tablebases using `chess.syzygy.Tablebase`. Hits replace engine evaluations with exact WDL (Win/Draw/Loss) values and the DTZ-optimal move as the policy target, providing mathematically perfect labels for the endgame.
4. **Target Computation:** Engine centipawn scores are converted to sparse top-20 policy distributions (stored as `uint16` move indices and `float16` scores) and normalised value targets (`float16`).
5. **Binary Sharding:** Processed positions are written into shards of 100K–500K positions in a compact binary format using packed bitboards (12 `uint64` values + 2 metadata bytes per position, approximately 98 bytes), enabling memory-mapped loading with zero deserialisation overhead.

Dataset Cleanup: Positions with analysis depth ≤ 10 or zero node counts are filtered. Decisively won positions ($|\text{cp}| > 1500$) and trivial mate-in-1/2 positions are downsampled to prevent value head bias. The

± 100 –500 centipawn range, where precise evaluation is most critical, is upsampled.

The resulting dataset occupies approximately 180–200 GB for one billion positions and loads via a `torch.utils.data.DataLoader` (map-style) with true random shuffling, `persistent_workers`, and configurable `prefetch_factor`.

3.4 Chess Implementation

Chess rules are not reimplemented; the framework delegates all legality checking, move generation, and game-over detection to the `python-chess` library. The `BoardWrapper` class provides a thin convenience layer over `chess.Board`, exposing properties such as `legal_moves_uci`, `is_checkmate`, and `halfmove_clock`. This delegation ensures correctness—`python-chess` handles edge cases such as en passant, castling through check, under-promotion, and the fifty-move rule—while keeping the framework’s codebase focused on machine learning concerns.

Board representation choices are determined by the encoder (Section 3.2.1) rather than by a single canonical format. This architecture allows each model family to receive the board in its most natural representation: a spatial tensor for CNNs, a graph for GNNs, and a token sequence for Transformers.

3.5 Web Platform Design

3.5.1 Platform Requirements

The web platform is designed to provide a browser-based interface for interacting with trained models. The core functional requirements are:

- A user must be able to play a full game of chess against any trained model (Human-vs-Bot mode).
- A user must be able to select two models and watch them play against each other in real time (Bot-vs-Bot mode).
- The interface must support model selection from all available checkpoints, displaying metadata such as architecture type and parameter count.
- Move updates must be streamed in real time with sub-second latency.

3.5.2 System Architecture

The platform follows a client-server architecture. The backend is implemented in Go, chosen for its lightweight goroutine-based concurrency model, which naturally maps to managing multiple simultaneous games without the overhead of thread pools. The frontend is built with Svelte, a compile-time reactive framework that produces minimal JavaScript bundles and efficient DOM updates—particularly suited to the frequent, localised state changes involved in rendering a chess board.

Communication between the frontend and backend uses WebSockets (RFC 6455) for bidirectional, low-latency move streaming. A REST API handles session management, model listing, and game creation, while the WebSocket channel carries move events and clock updates during active games.

3.5.3 Game Modes

- **Human-vs-Bot:** The user makes moves via the board UI; each move is sent to the backend, which invokes the selected model’s `get_move` method and streams the response back.
- **Bot-vs-Bot:** The backend orchestrates alternating `get_move` calls between two selected models, broadcasting each move to connected spectators with a configurable delay for readability.

3.5.4 User Interface Design

The interface centres on an interactive chessboard rendered using an SVG-based board component. Move input is handled via drag-and-drop with legal-move highlighting. A side panel displays the move list in standard algebraic notation, a real-time evaluation bar (when a value head is available), and model selection controls. The design prioritises clarity over decoration, following the conventions established by Lichess.

3.5.5 Backend API Design

The backend exposes the following core endpoints:

- `GET /api/models` — List available model checkpoints with metadata.
- `POST /api/games` — Create a new game session, specifying mode and model(s).
- `WS /api/games/{id}/ws` — WebSocket endpoint for real-time move streaming.
- `POST /api/games/{id}/move` — Submit a human move (Human-vs-Bot mode).

The backend loads PyTorch models via a Python subprocess bridge, communicating board states and receiving moves over a local socket. This isolates the Python ML runtime from the Go HTTP server, preventing GIL contention from affecting WebSocket responsiveness.

3.6 Technical Challenges

Several technical challenges were identified during design and addressed through targeted engineering decisions:

- **Graph Batching for GNNs:** Unlike tensor-based models, GNN inputs cannot be trivially stacked along a batch dimension because each graph has a variable number of edges. The collation function offsets edge indices by $i \times 64$ for the i -th graph in the batch and concatenates all edges into a single $(2, \sum E_i)$ tensor, with a `batch` vector mapping each node to its graph. This approach follows the PyTorch Geometric batching convention.
- **Variable-Length Transformer Sequences:** The PieceTransformer processes sequences of 1–32 tokens depending on the number of pieces on the board. A key-padding mask is constructed from the `attention_mask` field to prevent padded positions from influencing attention computation.
- **Mixed-Precision Training on Apple Silicon:** The MPS backend supports float16 autocast but does not support `GradScaler`. The trainer conditionally enables the scaler only on CUDA devices while still using autocast on MPS for throughput gains.
- **Soft vs. Hard Policy Labels:** Engine-annotated data provides centipawn scores for the top- N moves, enabling soft policy labels via a softmax over normalised scores. This carries more information than a one-hot best-move label but requires KL divergence rather than cross-entropy as the loss function. The pipeline supports both modes, selectable via configuration.

Chapter 4

Implementation

4.1 Development Environment and Tools

The framework is implemented in Python 3.10 and uses PyTorch as the primary deep learning backend. Table 4.1 summarises the principal libraries and their roles.

Table 4.1: Key libraries and their roles in the framework.

Library	Purpose
PyTorch	Neural network definition, training, and inference
PyTorch Geometric	Graph convolution and attention layers (GCN, GAT)
python-chess	Board representation, move generation, engine protocol
PyArrow	Parquet file I/O for the training dataset
NumPy / Pandas	Numerical operations and data analysis
Matplotlib	Evaluation flow charts and training curve visualisation
PyYAML	Configuration file parsing
tqdm	Progress bars for training and data generation

All experiments were conducted on two hardware configurations: an Apple M-series workstation using the MPS (Metal Performance Shaders) backend for local development and iteration, and an NVIDIA GPU cluster (CUDA) for large-scale training runs. The codebase auto-detects the best available device at startup, preferring CUDA over MPS over CPU, and enables hardware-specific optimisations accordingly (TF32 math and cuDNN auto-tuning on CUDA; float16 autocast on MPS).

Version control is managed with Git. All hyperparameters are specified in a single YAML configuration file (`config/default.yaml`), parsed at runtime into a hierarchy of Python dataclasses (`Config`, `ModelConfig`, `TrainingConfig`, etc.), ensuring that every experiment is fully reproducible from its configuration snapshot.

4.2 Core Framework Implementation

4.2.1 Abstract Base Classes

The framework’s extensibility rests on two abstract base classes:

- `ChessModel(ABC, nn.Module)`: Every neural network backbone subclasses this and implements `forward_backbone()` (returning a fixed-dimension feature vector) and `get_backbone_output_dim()`. The base class provides a concrete `forward(x)` that chains the backbone with whichever output head has been attached via `set_head()`.
- `StateEncoder(ABC)`: Every board encoder implements `encode(board)` and `get_input_shape()`. This contract allows the data pipeline and agents to work with any encoder–model pairing without type-checking.

4.2.2 Factory Pattern

The `create_model(config)` factory function reads the backbone name and head type from the configuration, instantiates the appropriate backbone with its architecture-specific parameters (e.g., number of residual blocks, number of attention heads), constructs the matching output head, and attaches it. A companion `get_encoder_for_model(backbone)` factory maps backbone names to their corresponding encoder classes. This two-step factory ensures that backbone–encoder pairings are always consistent.

4.2.3 Configuration System

All tuneable parameters are centralised in a YAML file and parsed into nested dataclasses:

- `HardwareConfig`: Device selection and data-loader worker count.
- `ModelConfig`: Backbone type, head type, and per-family sub-configs (`CNNConfig`, `TransformerConfig`, `GNNConfig`).
- `TrainingConfig`: Batch size, learning rate, weight decay, epoch count, loss weights, and LR scheduler type.
- `EngineConfig`: Path to the Stockfish binary and default search parameters.

A recursive `_dict_to_dataclass()` helper converts arbitrarily nested YAML dictionaries into their corresponding dataclass instances, providing type safety and IDE auto-completion throughout the codebase.

4.3 Supervised Learning Model

4.3.1 Data Preparation

Training data is sourced from the Lichess Stockfish evaluation dataset, which provides millions of positions annotated with engine analysis. The data is stored in Apache Parquet format, which offers efficient columnar compression and row-group-level random access.

The `ChessDataset` class, implemented as a PyTorch `IterableDataset`, supports two Parquet schemas:

1. **Engine-evaluated:** Each row contains a FEN string, the principal variation (PV) line in UCI notation, a centipawn evaluation, and an optional mate-in- N score. The first move of the PV line is used as the best move.
2. **Legacy multi-move:** Each row contains a FEN, the best move, the game result, and up to 20 alternative moves with their centipawn scores. This schema enables soft policy labels.

Data loading proceeds as follows:

1. The dataset enumerates all row-groups across all Parquet files.
2. Row-groups are partitioned across `DataLoader` workers by worker ID.
3. Within each worker, row-groups and rows are shuffled independently.
4. Each row is converted to a training example by encoding the board and building the policy and value targets.

Policy targets are constructed in one of two modes. In *soft label* mode, the centipawn scores of the top- N moves are normalised by their absolute maximum, passed through a softmax, and scattered into a sparse 4544-dimensional vector. In *hard label* mode, the best move receives a probability of 1.0.

Value targets are derived from the strongest available signal: the game result (+1, 0, -1), the mate score (mapped to ± 1), or the centipawn evaluation (divided by 1000 and clamped to $[-1, 1]$). The value is always expressed from the current player’s perspective.

A custom `collate_fn` batches examples into the format expected by each model family. For CNN models, input tensors are stacked along the batch dimension. For Transformer models, each dictionary field (tokens, positions, masks) is stacked independently. For GNN models, edge indices are concatenated with per-graph offsets of 64 nodes, and a `batch` vector is constructed to map each node to its originating graph—following the PyTorch Geometric batching convention.

4.3.2 Neural Network Architecture

Six backbone architectures are implemented, two per model family.

4.3.2.1 Convolutional Neural Networks

ConvNet consists of six sequential convolutional blocks, each comprising a 3×3 convolution (with padding to preserve spatial dimensions), batch normalisation, and ReLU activation. The first block projects the 18-channel input to 256 channels; subsequent blocks maintain this width. A global average pooling layer reduces the $256 \times 8 \times 8$ feature maps to a 256-dimensional vector.

ResNet begins with an initial 3×3 convolution that projects the input to 256 channels, followed by N residual blocks (configurable to 6, 10, or 20). Each residual block contains two 3×3 convolutions with batch normalisation, and the input is added to the output via a skip connection before the final ReLU. Global average pooling produces the output vector. The skip connections stabilise training in deeper configurations by providing gradient shortcuts.

4.3.2.2 Transformers

SquareTransformer tokenises the board as 64 fixed tokens (one per square) using a vocabulary of 13 (empty + 12 piece types). Each token is embedded and summed with a learnable positional embedding and a coordinate projection (normalised rank and file). Side-to-move information is broadcast as a bias to all tokens, while castling rights are injected into a prepended CLS token. The sequence passes through N Transformer encoder blocks, each using pre-LayerNorm, multi-head self-attention (8 heads), and a GELU-activated feed-forward network with a $4\times$ expansion ratio. The CLS token’s final hidden state serves as the backbone output.

PieceTransformer tokenises the board as a variable-length sequence of up to 32 tokens (one per piece). Each token encodes the piece type (vocabulary of 12) and its square position via separate embedding tables, plus a sequence-order embedding. A key-padding mask prevents attention from flowing to padded positions. The architecture otherwise mirrors the SquareTransformer, with a CLS token aggregating the sequence representation.

4.3.2.3 Graph Neural Networks

GCN projects the 14-dimensional node features to a hidden dimension of 256 via a linear layer, then applies N GCN layers. Each layer performs a graph convolution (**GCNConv**), batch normalisation, and ReLU activation with a residual connection. A global mean pooling aggregates node embeddings into a single graph-level vector,

which is summed with a side-to-move embedding and a castling projection before a final linear output layer.

GAT follows the same overall structure but replaces **GCNConv** with **GATConv**, which computes multi-head attention (4 heads) over neighbours. Critically, the GAT layers accept the 3-dimensional edge attributes (distance, ray-tracing, legality) via the `edge_dim` parameter, allowing the attention mechanism to weight edges based on their tactical significance. ELU activation is used in place of ReLU for smoother gradients in the attention computation.

4.3.3 Training Process

4.3.3.1 Loss Functions

Three loss modules are provided:

- **PolicyLoss:** KL divergence for soft labels ($\mathcal{L}_{\text{KL}} = \sum_i t_i \log \frac{t_i}{p_i}$, where t is the target distribution and $p = \text{softmax}(\text{logits})$) or cross-entropy for hard labels, with optional label smoothing.
- **ValueLoss:** Mean squared error between the predicted value (tanh-bounded) and the target.
- **DualLoss:** Weighted sum $\mathcal{L} = \lambda_p \mathcal{L}_{\text{policy}} + \lambda_v \mathcal{L}_{\text{value}}$.

4.3.3.2 Optimisation

The AdamW optimiser is used with a default learning rate of 10^{-3} and weight decay of 10^{-4} . Learning rate scheduling supports cosine annealing (default) and step decay. Gradients are clipped to a maximum norm of 1.0 to prevent instability during early training.

4.3.3.3 Mixed Precision and Compilation

On CUDA devices, training uses float16 autocast with **GradScaler** to maintain numerical stability. On MPS (Apple Silicon), autocast is enabled but the scaler is disabled as MPS does not support it. Non-GNN backbones are wrapped with `torch.compile` for kernel fusion and operator-level optimisation. GNN models are excluded from compilation due to dynamic graph structures that prevent effective tracing.

4.3.3.4 Checkpointing

After each epoch, the trainer serialises the model state dict, optimiser state, scheduler state, epoch number, global step count, and best validation loss. At the end of training, intermediate checkpoints are pruned to retain only every fifth epoch, reducing storage overhead while preserving sufficient granularity for post-hoc analysis.

4.4 Architectural Enhancements

Following the initial implementation and early training runs, several architectural improvements were introduced to address identified weaknesses.

4.4.1 Enhanced CNN Encoder

The base 18-channel encoder was extended to 32 channels. The additional 14 planes are computed from the `python-chess` board state at encoding time:

- **Attack/Defence Maps (4 planes):** For each side, per-square attacker and defender counts are computed using `board.attackers()` and normalised by their maximum value across the board.
- **Mobility (2 planes):** For each piece, the number of legal moves is counted and broadcast to its square. A separate plane is generated for each side.
- **Pin Detection (1 plane):** Absolute pins to the King are detected by temporarily removing each friendly piece and checking whether the King becomes attacked along the vacated ray.
- **King Safety (2 planes):** A heuristic score is computed per side based on pawn shield integrity (presence of pawns on the second and third ranks adjacent to the King) and open files near the King.
- **Material Imbalance (5 planes):** Per-piece-type count differences (Pawn, Knight, Bishop, Rook, Queen) are computed and broadcast as constant 8×8 planes.

4.4.2 Spatial Policy Head

The initial policy head used global average pooling, collapsing the $C \times 8 \times 8$ feature maps into a C -dimensional vector before projecting to the 4544 move logits. This discards spatial locality. The spatial policy head replaces GAP with a 1×1 convolution that reduces the channel count to C' , followed by `flatten()`, producing a $C' \times 8 \times 8 = 64C'$ vector. A final linear layer maps this to 4544 logits. This allows the network to associate specific board squares with specific move origins and destinations.

4.4.3 Squeeze-and-Excitation Blocks

SE blocks were inserted into each `ResidualBlock`. After the second convolution and batch normalisation, the feature maps are squeezed via global average pooling to a channel vector of dimension C , passed through a bottleneck (`Linear( $C$ ,  $C/r$ )` $\rightarrow$ `ReLU` $\rightarrow$ `Linear( $C/r$ ,  $C$ )` $\rightarrow$ `Sigmoid`, with reduction ratio $r = 16$), and

the resulting channel weights multiply the original feature maps. Because the SE block sits inside the residual path, the skip connection ensures stable gradient flow even in deep (20-block) configurations.

4.4.4 Flash Attention

The Transformer encoder blocks were updated to use `torch.nn.functional.scaled_dot_product_attention` in place of the explicit QKV matmul and softmax. On supported hardware, this dispatches to the Flash Attention kernel, reducing attention memory from $O(n^2)$ to $O(n)$ and providing a measurable training speedup for the 64-token SquareTransformer sequence length.

4.4.5 Activation Functions

ReLU activations in the ConvNet and ResNet backbones were replaced with SiLU (Swish), which provides smoother gradients near zero. The Transformer blocks retained GELU, which was used from the initial implementation.

4.5 Data Pipeline Evolution

The initial streaming Parquet pipeline (Section 4.5.1) was effective for prototyping but became the training bottleneck at scale due to per-row `python-chess` encoding. A five-stage offline pipeline was implemented to support over one billion positions.

4.5.1 Stage 1: Deduplication

DuckDB’s out-of-core SQL engine is used to normalise FEN strings (stripping halfmove and fullmove counters), group by the normalised FEN, and retain only the row with the highest analysis depth or node count. This eliminates redundant positions that would otherwise bias the training distribution toward common openings.

4.5.2 Stage 2: PV Line Unrolling

For each base position, the first N moves of the principal variation are played out using `python-chess`. Each intermediate position is emitted as a separate training example, with the subsequent PV move as its policy target. The root evaluation is inherited with sign flips for alternating sides, avoiding the prohibitive cost of re-evaluating every unrolled position with Stockfish. A drift analysis validated this approximation: 96% of inherited evaluations remained within ± 100 centipawns of the independently computed value.

4.5.3 Stage 3: Tablebase Enrichment

Positions with five or fewer pieces are probed against the Syzygy 5-piece tablebases (approximately 1 GB on disk) using `chess.syzygy.Tablebase`. On a hit, the engine’s approximate evaluation is replaced with the mathematically exact WDL outcome (+1, 0, or −1), and the policy target is set to the DTZ-optimal (Distance To Zeroing) move. At inference time, the agent bypasses the neural network entirely for ≤ 5 -piece positions, playing the tablebase-optimal move directly.

4.5.4 Stage 4: Target Computation and Cleanup

Engine centipawn scores are converted to sparse policy distributions: only the top 20 moves are stored as pairs of `uint16` indices and `float16` scores (81 bytes per position). Value targets are stored as `float16` (2 bytes).

Dataset cleanup rules are applied:

- Positions with analysis depth ≤ 10 or zero node counts are removed.
- Decisively won positions ($|\text{cp}| > 1500$) and trivial mate-in-1/2 are downsampled to prevent value head bias toward extreme evaluations.
- The ± 100 –500 centipawn range, where precise evaluation is most critical for playing strength, is upsampled.

4.5.5 Stage 5: Binary Sharding

Processed positions are packed into binary shards of 100K–500K positions. Each position is stored as 12 `uint64` bitboards plus 2 metadata bytes (approximately 98 bytes for the board state), alongside the sparse policy and value targets. The resulting dataset occupies approximately 180–200 GB for one billion positions and is loaded via a map-style `torch.utils.data.Dataset` with memory-mapped access, true random shuffling via `DistributedSampler`, `persistent_workers=True`, and configurable `prefetch_factor`.

4.6 Baseline Models

Two baselines anchor the evaluation:

- **RandomAgent:** Plays uniformly at random from the legal move set. This provides a lower bound on playing strength and verifies that trained models have learned meaningful patterns.

- **UCIAgent (Stockfish):** Wraps Stockfish at configurable depth and skill levels. At full strength (depth 15+), Stockfish serves as the upper bound. At reduced depths (1–5), it provides intermediate calibration points for estimating ELO.

4.7 Benchmarking and Evaluation Pipeline

The benchmarking system automates model evaluation through the following pipeline:

1. **Game Execution:** Each trained model plays a configurable number of games against Stockfish, alternating colours for fairness. Both raw-policy (greedy argmax) and MCTS-augmented agents are supported.
2. **Per-Move Evaluation:** After every move, Stockfish analyses the resulting position at depth 18, recording the centipawn score from White’s perspective. Mate scores are converted to large constants (± 10000).
3. **PGN Export:** Each game is exported in Portable Game Notation with embedded evaluation comments, enabling review in standard chess software (Lichess, Chess.com, ChessBase).
4. **Visualisation:** Matplotlib generates evaluation flow charts for each game, plotting centipawn score against move number with win/draw/loss shading. A combined comparison plot overlays all models.
5. **Timing:** Per-move inference times are recorded for both the model and the Stockfish opponent, enabling latency comparisons.
6. **Reporting:** A Markdown report is generated with win/draw/loss tables, timing statistics (mean, median, min, max), and links to PGN and figure files.

4.8 Testing and Debugging

Unit tests cover critical components including move encoding (verifying that all 4544 UCI moves round-trip through the index lookup), encoder output shapes, model forward-pass dimensions, and loss function gradients. Integration tests verify end-to-end training on a small synthetic dataset to catch dimension mismatches across the encoder–model–head pipeline.

Debug utilities include the `CNNEncoder.decode_piece_planes()` method, which reconstructs piece positions from an encoded tensor for visual inspection, and the `LearningAgent.get_move_distribution()` method, which returns the top- k moves with their softmax probabilities for diagnosing policy quality.

4.9 Documentation

The codebase follows a consistent documentation standard: every module, class, and public method includes a docstring specifying its purpose, arguments, and return values. A `README.md` provides installation instructions, a quick-start guide, and example training commands. The `setup.sh` script automates environment creation and dependency installation.

Chapter 5

Evaluation

5.1 Evaluation Methodology

5.2 Supervised Learning Model Results

5.2.1 Training Performance

5.2.2 Playing Strength

5.2.3 Error Analysis

5.3 Comparative Analysis

5.4 Ablation Studies

5.5 Web Platform Evaluation

5.5.1 Usability Testing

5.5.2 Performance and Responsiveness

5.5.3 Bot-vs-Bot Match Results via Platform

5.6 Comparison with Related Work

5.7 Interpretation and Significance

5.8 Limitations and Challenges

Chapter 6

Critical Review and Reflection

6.1 Achievement of Objectives

The primary objective of this project was to engineer and evaluate a suite of deep learning models capable of playing chess without search algorithms. The framework successfully implements all six proposed architectures (ConvNet, ResNet, SquareTransformer, PieceTransformer, GCN, GAT), each trainable under identical data conditions with interchangeable output heads. The modular backbone-head design and the streaming data pipeline both exceed the original specification, which anticipated only three architectures.

The secondary objectives of inference speed measurement and cross-architecture comparison are fully supported by the benchmarking pipeline, which records per-move timing and exports PGN files for qualitative analysis.

Beyond the initial architecture, the framework evolved substantially during development. The CNN encoder was extended from 18 to 32 channels with tactical feature planes (attack/defence maps, mobility, pins, king safety) and per-piece-type material imbalance encoding. The ResNet backbone was augmented with Squeeze-and-Excitation blocks for dynamic channel attention. A spatial policy head replaced global average pooling to preserve square-level information. The Transformer models adopted Flash Attention via `scaled_dot_product_attention`. Most significantly, the data pipeline was redesigned from a streaming Parquet reader to a five-stage offline system (DuckDB deduplication, PV line unrolling, Syzygy tablebase enrichment, target computation, and binary sharding) capable of handling over one billion positions.

6.2 What Worked Well

- **Backbone-Head Decoupling:** The abstract `ChessModel` interface proved highly effective. Adding the GAT architecture, for instance, required implementing only a single class with two methods; the training

loop, loss functions, and evaluation pipeline required zero modifications.

- **Streaming Data Pipeline:** The row-group-level streaming from Parquet files allowed training on datasets larger than available RAM without any changes to the training loop. This was particularly valuable when scaling from initial prototyping (100K positions) to full-scale runs (tens of millions of positions).
- **YAML Configuration:** Centralising all hyperparameters in a single file eliminated hard-coded constants and made experiment management straightforward. Combined with checkpoint serialisation, any past experiment can be exactly reproduced.
- **Soft Policy Labels:** Using softmax distributions over the engine’s top- N move evaluations rather than one-hot best-move labels provided richer gradient information, particularly for positions where multiple moves are nearly equally strong.
- **PV Unrolling with Inherited Evaluations:** The decision to inherit root evaluations (with sign flips) for PV-unrolled positions rather than re-evaluating each with Stockfish proved both practical and accurate. The drift analysis showing 96% of inherited values within $\pm 100\text{cp}$ validated this approximation and enabled a dramatic increase in training data diversity at negligible computational cost.
- **Tactical Feature Planes:** Augmenting the CNN encoder with explicit attack/defence maps, mobility, and pin detection planes provided the model with domain knowledge that would otherwise require many convolutional layers to infer. This was particularly impactful for the shallower ConvNet, where the limited receptive field makes it difficult to learn long-range piece interactions from raw bitboards alone.
- **Syzygy Tablebase Integration:** Using mathematically perfect endgame labels for ≤ 5 -piece positions eliminated engine approximation error in the training signal for the endgame regime, and the inference-time bypass ensured flawless endgame play without any neural network involvement.

6.3 What Didn’t Work Well

- **GNN Encoding Speed:** The GNN encoder is significantly slower than the CNN encoder because it must compute attack and defence relations for every square on every board. This makes the GNN data pipeline the bottleneck during training, despite the model itself being no larger than the CNN.
- **Initial Global Average Pooling for Policy:** The original policy head used global average pooling, discarding spatial information critical for move prediction. This was identified early and replaced with a spatial policy head, but the initial training runs on the pooled representation produced noticeably weaker policy accuracy, confirming that square-level information is essential for move selection.

6.4 Lessons Learned

- **Data quality matters more than data quantity.** Filtering out low-depth evaluations and balancing the value distribution had a larger impact on model performance than simply increasing dataset size.
- **Encoder design is as important as model architecture.** The choice of board representation (spatial tensor vs. graph vs. token sequence) determines what information is accessible to the model and what must be learned from scratch.
- **Start with the simplest architecture.** The ConvNet, despite being the simplest model, served as an invaluable debugging tool. Once the ConvNet was training correctly, issues in more complex architectures (Transformer attention masks, GNN edge batching) could be diagnosed by comparison.
- **Mixed-precision training requires platform-specific handling.** The asymmetry between CUDA and MPS support for `GradScaler` required conditional logic that was non-obvious and initially caused silent numerical instability.

6.5 Personal Reflection

This project significantly deepened my understanding of deep learning engineering beyond the classroom. Implementing graph neural networks from scratch—including the non-trivial batching of variable-edge graphs—required engaging with PyTorch Geometric at a level far beyond its tutorial examples. The Transformer implementations reinforced the importance of positional encoding design: absolute, relative, and learnable encodings produced noticeably different training dynamics on the same data.

Managing a multi-architecture codebase also developed my software engineering discipline. The abstract base class pattern, which initially felt like over-engineering for a single-developer project, proved essential when adding the sixth architecture three months into development.

6.6 Future Work

6.6.1 Reinforcement Learning

The framework currently relies entirely on supervised learning from engine-annotated data. Two reinforcement learning paradigms are natural extensions:

- **MCTS Self-Play:** The MCTSAgent already implemented in the framework could be used to generate self-play training data, following the AlphaZero paradigm. Each game would record MCTS visit-count

distributions as policy targets and game outcomes as value targets, enabling the model to improve beyond the ceiling of its training data.

- **Curriculum Stockfish RL:** Playing the model against Stockfish at progressively increasing difficulty levels (varying skill level and search depth) would provide a structured reinforcement signal. A curriculum from beginner (skill 0, depth 1) to club level (skill 5, depth 5) would allow the model to learn incrementally from opponents matched to its current strength.

6.6.2 Web Platform

The planned Go/Svelte web platform would provide a browser-based interface for interacting with trained models:

- **Human-vs-Bot:** Users play against any trained model via a drag-and-drop board interface with real-time move streaming over WebSockets.
- **Bot-vs-Bot:** Two models play each other with spectator support, configurable move delay, and live evaluation bars.
- **Tournament Mode:** A round-robin system where all trained models play against each other, with automated ELO calculation from the results.
- **Analysis Board:** An interactive mode for setting up arbitrary positions and comparing move recommendations of different models side by side.

6.6.3 Long-term Research Directions

- **Sequence Modelling:** Framing chess as a sequence prediction task—analogueous to language modelling—where the model autoregressively predicts move continuations from a history of board states. This Decision Transformer-style approach could enable genuine multi-move planning without explicit search.
- **Transfer to Other Games:** The backbone-head architecture is game-agnostic. Adapting the framework to other perfect-information games (e.g., Shogi, Othello) would validate the generality of the design.
- **6- and 7-Piece Tablebases:** Extending the tablebase integration to larger piece counts (at the cost of significantly more storage) would expand the range of positions with perfect play.

6.7 Broader Impact

The framework contributes an open-source, controlled benchmark for comparing neural network architectures on a shared chess dataset—a resource that, to the best of our knowledge, does not exist in the published literature. The modular design lowers the barrier to entry for researchers who wish to experiment with novel architectures or training regimes without reimplementing the full chess AI stack. The web platform, once complete, will provide an accessible demonstration tool for communicating deep learning concepts to non-specialist audiences.

Chapter 7

Legal, Social, Ethical, and Professional Issues

7.1 Ethical Considerations

7.1.1 Research Ethics

This project does not involve human participants, personal data collection, or experimentation on living subjects. Ethical approval was obtained through the University of Surrey’s SAGE (Self-Assessment for Governance and Ethics) process, which confirmed that the project falls within the low-risk category. All training data is derived from publicly available chess game databases and open-source engine evaluations.

7.1.2 Responsible AI

The models developed in this project are narrow AI systems designed exclusively for chess move prediction and position evaluation. They pose no direct risk of harm. However, the broader principle of transparency in AI decision-making is relevant: the framework’s evaluation pipeline (centipawn tracking, PGN export) provides full traceability of every move decision, and the soft policy labels expose the model’s uncertainty across candidate moves. This level of interpretability exceeds that of most commercial chess engines, which return only their top move without a probability distribution.

Bias in training data is a consideration: the Lichess Stockfish evaluation dataset reflects the engine’s playing style and evaluation heuristics. Models trained on this data will inherit Stockfish’s strategic preferences (e.g., material-focused evaluation), which may not align with alternative playing styles. This is an acceptable limitation given the project’s goal of knowledge distillation from a strong engine.

7.2 Legal Issues

7.2.1 Intellectual Property

The framework is developed as original work for the University of Surrey Final Year Project and complies with the University’s intellectual property policy. No proprietary code or datasets are used.

7.2.2 Copyright and Licensing

All third-party libraries are used in compliance with their respective open-source licences:

- `python-chess`: GPL-3.0 — permits use, modification, and distribution.
- PyTorch: BSD-3-Clause — permissive licence with minimal restrictions.
- PyTorch Geometric: MIT — permissive licence.
- Stockfish: GPL-3.0 — used as an external binary for evaluation; the framework does not link against or modify its source code.
- Lichess game data: Creative Commons CC0 — public domain, no restrictions on use.

The training dataset is derived from Lichess’s publicly available database, which is released under Creative Commons CC0 (public domain). The Stockfish evaluations annotating this data are produced by an open-source engine and carry no additional copyright restrictions.

7.3 Social Issues

7.3.1 Accessibility

The framework is implemented in Python, the most widely taught programming language in computer science education, and uses well-documented libraries. The YAML configuration system and command-line training scripts are designed to be accessible to users with basic Python proficiency. The requirement for a GPU for practical training times is a barrier for some users, though the framework functions correctly on CPU.

7.3.2 Web Platform Accessibility

The planned web platform targets modern evergreen browsers (Chrome, Firefox, Safari, Edge) and is designed to be responsive across desktop and tablet screen sizes. The SVG-based board rendering ensures crisp display

at any resolution. Keyboard accessibility for move input and ARIA labels for screen readers are planned but not yet implemented.

7.3.3 Impact on Chess Community

The project contributes to the growing body of open-source chess AI research. By providing a controlled multi-architecture comparison and an accessible web interface, it may serve as an educational resource for players interested in understanding how neural networks evaluate positions. The models are not strong enough to be used for cheating in competitive play, and the framework includes no mechanisms for real-time assistance during online games.

7.4 Professional Issues

7.4.1 Code of Conduct

Development adheres to the BCS (British Computer Society) Code of Conduct, with particular attention to:

- **Public interest:** The framework is designed as an open educational and research tool.
- **Professional competence:** The codebase follows established software engineering practices including version control, automated testing, comprehensive documentation, and modular design.
- **Duty to the profession:** All results are reported honestly, including negative results and known limitations.

7.4.2 Information Security

The framework processes no personal data and stores no user credentials. The web platform’s backend API validates all inputs (FEN strings, move UCI strings) before processing. No sensitive information (API keys, passwords) is hard-coded in the repository.

7.4.3 Web Platform Security

The planned web platform mitigates common web vulnerabilities:

- **Input Validation:** All FEN strings and UCI moves received from clients are validated against `python-chess` before execution.

- **WebSocket Security:** Connections use WSS (WebSocket Secure) in production deployments.
- **Rate Limiting:** Move submission endpoints are rate-limited to prevent denial-of-service via rapid move flooding.

7.5 Sustainability

7.5.1 Environmental Impact

Neural network training is computationally intensive and carries a non-trivial carbon footprint. The following measures reduce the environmental impact of this project:

- **Mixed-precision training** halves the memory bandwidth and increases throughput by up to $2\times$, reducing the wall-clock time (and energy consumption) of each training run.
- **Streaming data loading** avoids redundant I/O by reading each row-group exactly once per epoch.
- **Checkpoint pruning** reduces storage overhead by retaining only every fifth epoch.
- **Search-less inference** is the project’s raison d’être: a single forward pass consumes orders of magnitude less energy than the millions of node evaluations performed by a searching engine.

7.5.2 UN Sustainable Development Goals

This project aligns with the following UN Sustainable Development Goals:

- **SDG 4 (Quality Education):** The framework and web platform serve as educational tools for understanding deep learning architectures in an engaging, interactive context.
- **SDG 9 (Industry, Innovation, and Infrastructure):** The project advances AI research by providing the first controlled cross-architecture comparison of CNN, Transformer, and GNN models on identical chess data.
- **SDG 12 (Responsible Consumption and Production):** The emphasis on computational efficiency (mixed precision, search-less inference) reflects a commitment to minimising resource consumption.

7.6 Summary

This project adopts a responsible development posture: it uses open-source tools and public-domain data, produces transparent and interpretable outputs, processes no personal information, and actively minimises its

computational footprint. No significant ethical, legal, or social risks have been identified.

Chapter 8

Conclusion

This project set out to investigate whether deep learning architectures can develop meaningful chess-playing ability without the safety net of search algorithms. To this end, a modular framework was engineered that supports six neural network backbones—ConvNet, ResNet, SquareTransformer, PieceTransformer, GCN, and GAT—each trainable under identical conditions with interchangeable policy, value, and dual output heads.

The framework makes several contributions. First, it provides the first controlled, cross-architecture comparison of CNN, Transformer, and GNN models trained on the same Lichess Stockfish evaluation data with consistent preprocessing, loss functions, and evaluation protocols. Second, it demonstrates that knowledge distillation from a strong engine via soft policy labels is a viable and scalable training signal for search-less models.

The project evolved substantially beyond its initial design. The CNN encoder was extended from 18 to 32 channels with tactical feature planes (attack/defence maps, mobility, pin detection, king safety) and per-piece-type material imbalance encoding, injecting domain knowledge that would otherwise require many layers to learn. The ResNet backbone was augmented with Squeeze-and-Excitation blocks for dynamic channel attention, and a spatial policy head replaced global average pooling to preserve square-level information critical for move selection. The Transformer models adopted Flash Attention for improved memory efficiency and training throughput.

Most significantly, the data pipeline was redesigned from a streaming Parquet reader into a five-stage offline system capable of handling over one billion positions. DuckDB-based deduplication removes redundant positions, PV line unrolling generates diverse intermediate positions with inherited evaluations (validated at 96% accuracy within ± 100 cp), and Syzygy tablebase enrichment provides mathematically perfect labels for endgame positions with five or fewer pieces. The resulting binary-sharded dataset, with packed bitboard storage and sparse policy representations, enables efficient memory-mapped training at scale.

From a software engineering perspective, the backbone-head decoupling pattern, the factory-based model cre-

ation, and the YAML-driven configuration system collectively lower the barrier to experimentation: adding a new architecture requires implementing a single abstract class, while all training, evaluation, and benchmarking infrastructure is reused without modification.

The project acknowledges several limitations. Without search, the models are expected to struggle with deep tactical sequences (the “horizon effect”) and highly technical endgames beyond the tablebase’s piece-count threshold. The GNN encoder’s computational cost, driven by per-position attack relation computation, remains a training throughput bottleneck despite the offline pre-encoding pipeline addressing this for CNN and Transformer models.

Looking ahead, reinforcement learning via MCTS self-play and curriculum-based Stockfish RL represents the most promising direction for pushing model strength beyond the supervised learning ceiling. The planned Go/Svelte web platform would provide an accessible interface for human-vs-bot and bot-vs-bot interaction, and the exploration of sequence modelling approaches (Decision Transformer-style autoregressive move prediction) could enable genuine multi-move planning without explicit search.

In summary, this project demonstrates that the “search-less chess” paradigm is not only technically feasible but also a fertile ground for comparing how different neural network families internalise spatial, relational, and sequential structure. The framework—with its enhanced encoders, attention mechanisms, and billion-position data pipeline—provides the infrastructure for this ongoing investigation.

Appendix A

Code Listings

Appendix B

Additional Results

Appendix C

User Guide

Appendix D

Game Examples

Appendix E

Ethical Approval Documentation

Appendix F

Project Planning Documents

Bibliography

- [1] J. A. Briffa, H. G. Schaathun, and S. Wesemeyer, “An improved decoding algorithm for the Davey-MacKay construction,” in *Proc. IEEE Intern. Conf. on Commun.*, Cape Town, South Africa, May 23–27, 2010.
- [2] A. A. el Gamal, L. A. Hemachandra, I. Shperling, and V. K. Wei, “Using simulated annealing to design good codes,” *IEEE Transactions on Information Theory*, vol. 33, no. 1, pp. 116–123, Jan. 1987.
- [3] P. G. Farrell, “Notes on source coding,” Feb. 1992, university of Manchester.
- [4] B. P. Tunstall, “Synthesis of noiseless compression codes,” Ph.D. dissertation, Georgia Institute of Technology, 1968.
- [5] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes in C: The Art of Scientific Computing*, 2nd ed. Cambridge University Press, 1992.
- [6] B. Henderson, *Netpbm*, Feb. 22nd, 2009. [Online]. Available: <http://netpbm.sourceforge.net/doc/>
- [7] *NVIDIA CUDA Programming Guide*, NVIDIA Corporation, Jul. 1st, 2009, version 2.3.
- [8] M. Burrows and D. J. Wheeler, “A block-sorting lossless data compression algorithm,” Digital SRC Research Report, Tech. Rep., 1994.